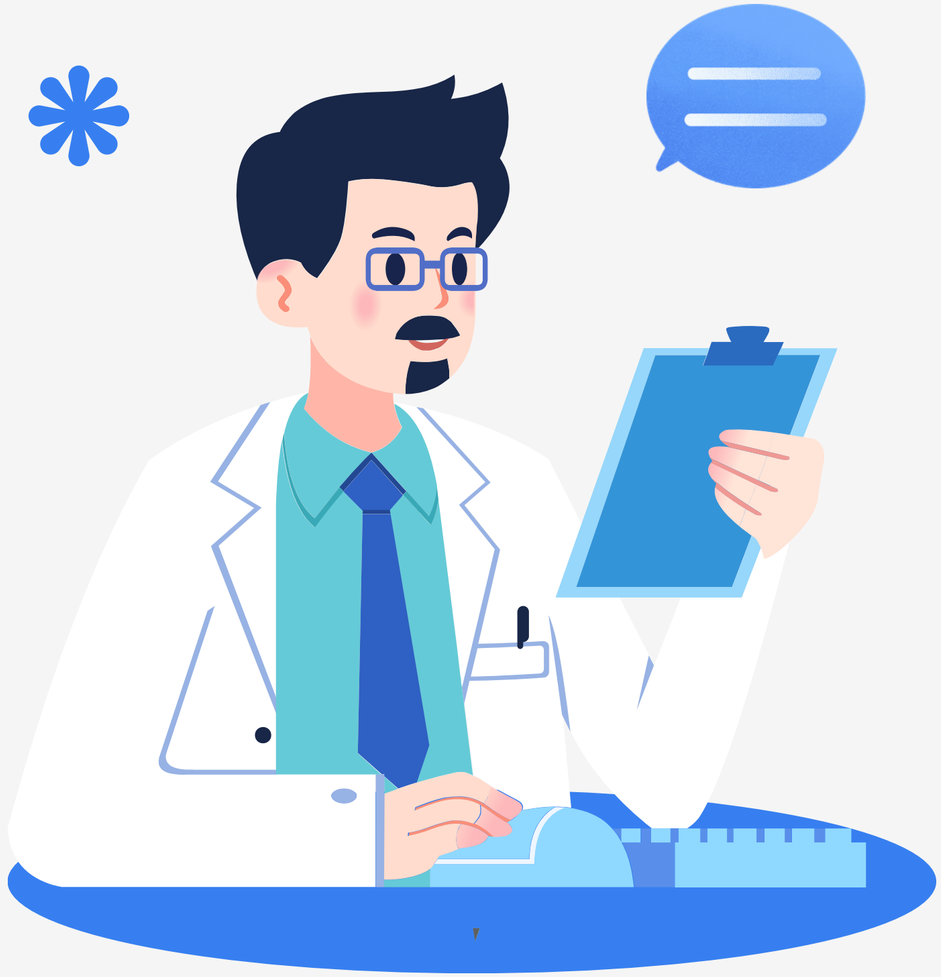


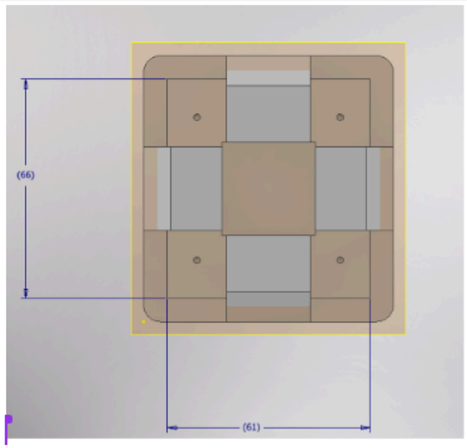
CASO CLÍNICO 5

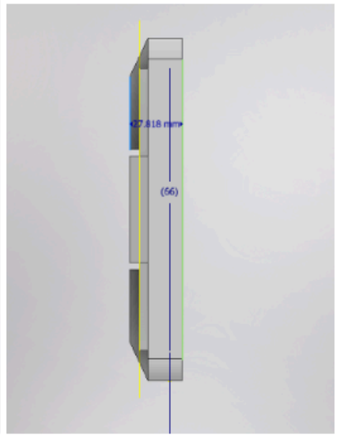
EQUIPO 4

Implementación preliminar de los subsistemas

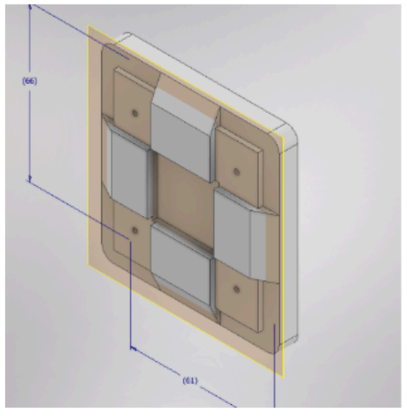


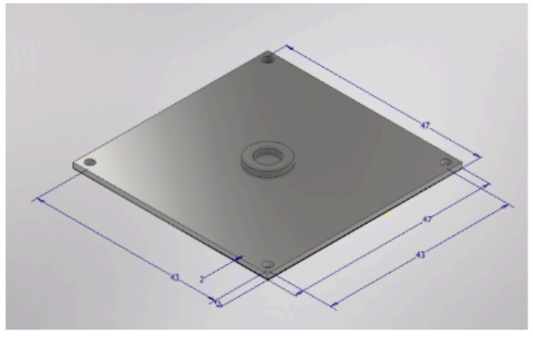
Mecánica y fabricación

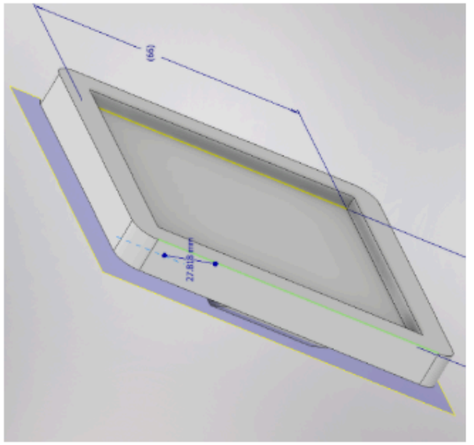






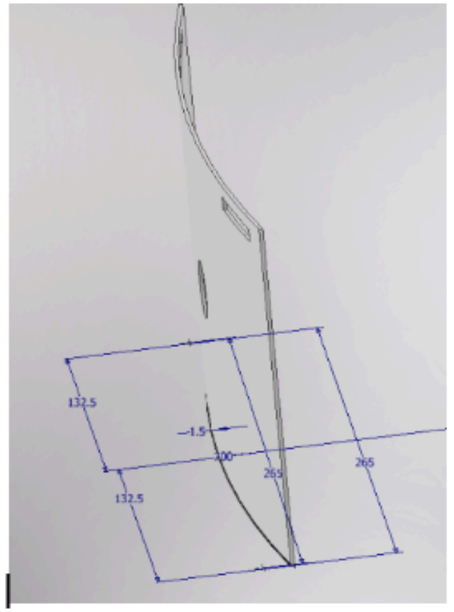


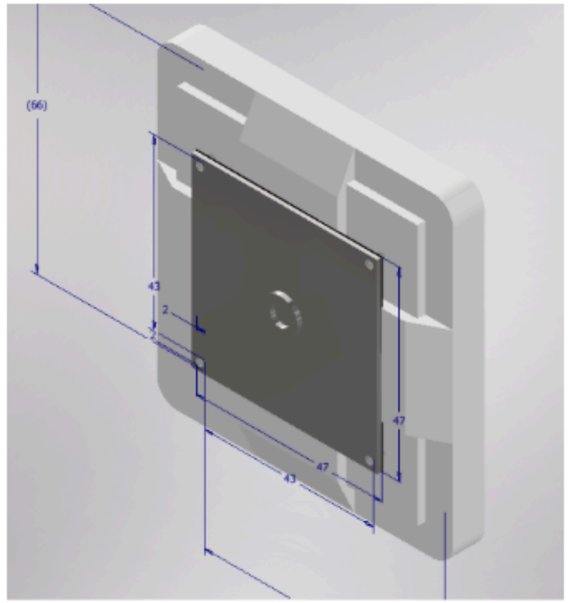


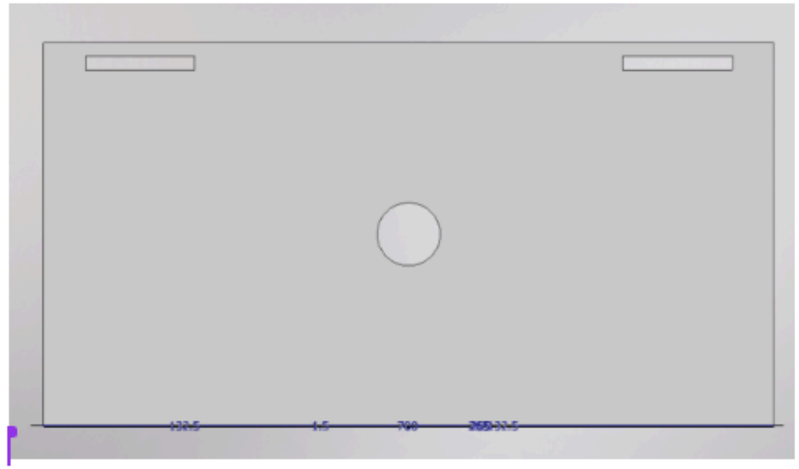


Implementación preliminar de los subsistemas

Mecánica y fabricación

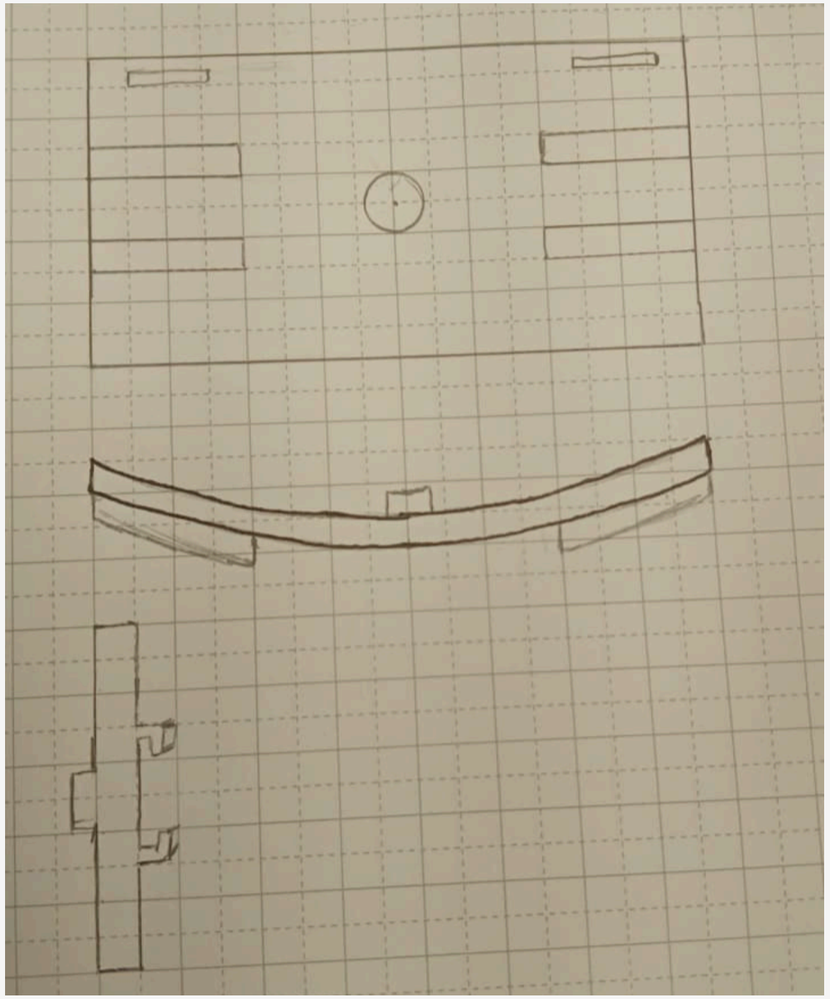






Implementación preliminar de los subsistemas

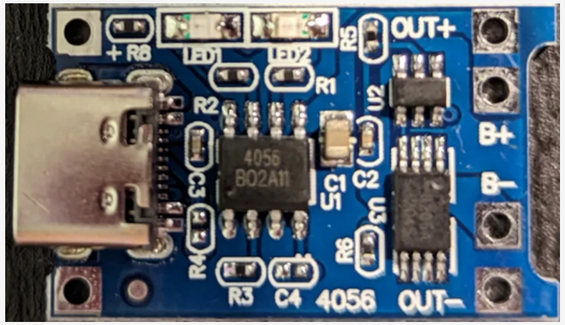
Mecánica y fabricación



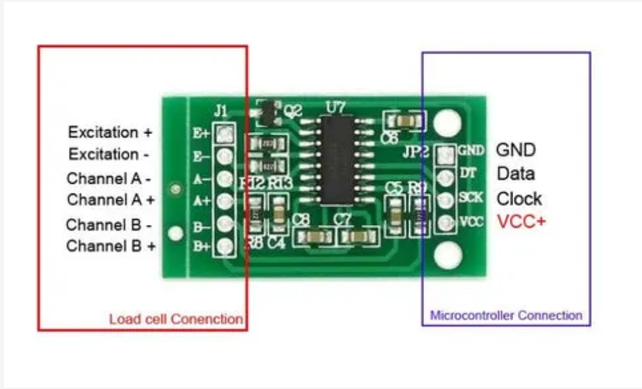
Implementación preliminar de los subsistemas

Electronica y sensores

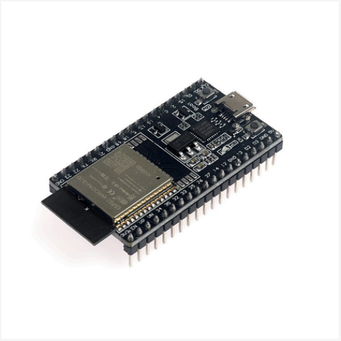




tp4056 type c



XH711 module



ESP32 (DOIT ESP32 DEVKIT V1)

Implementación preliminar de los subsistemas

Electronica y sensores



Conector PH de 4 pines



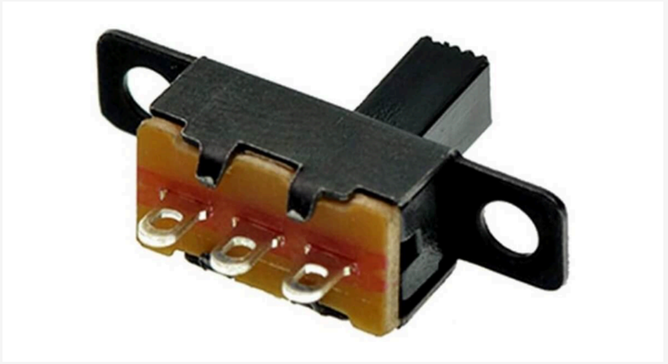
Condensador de 100nF



Resistencia de 100 Ω



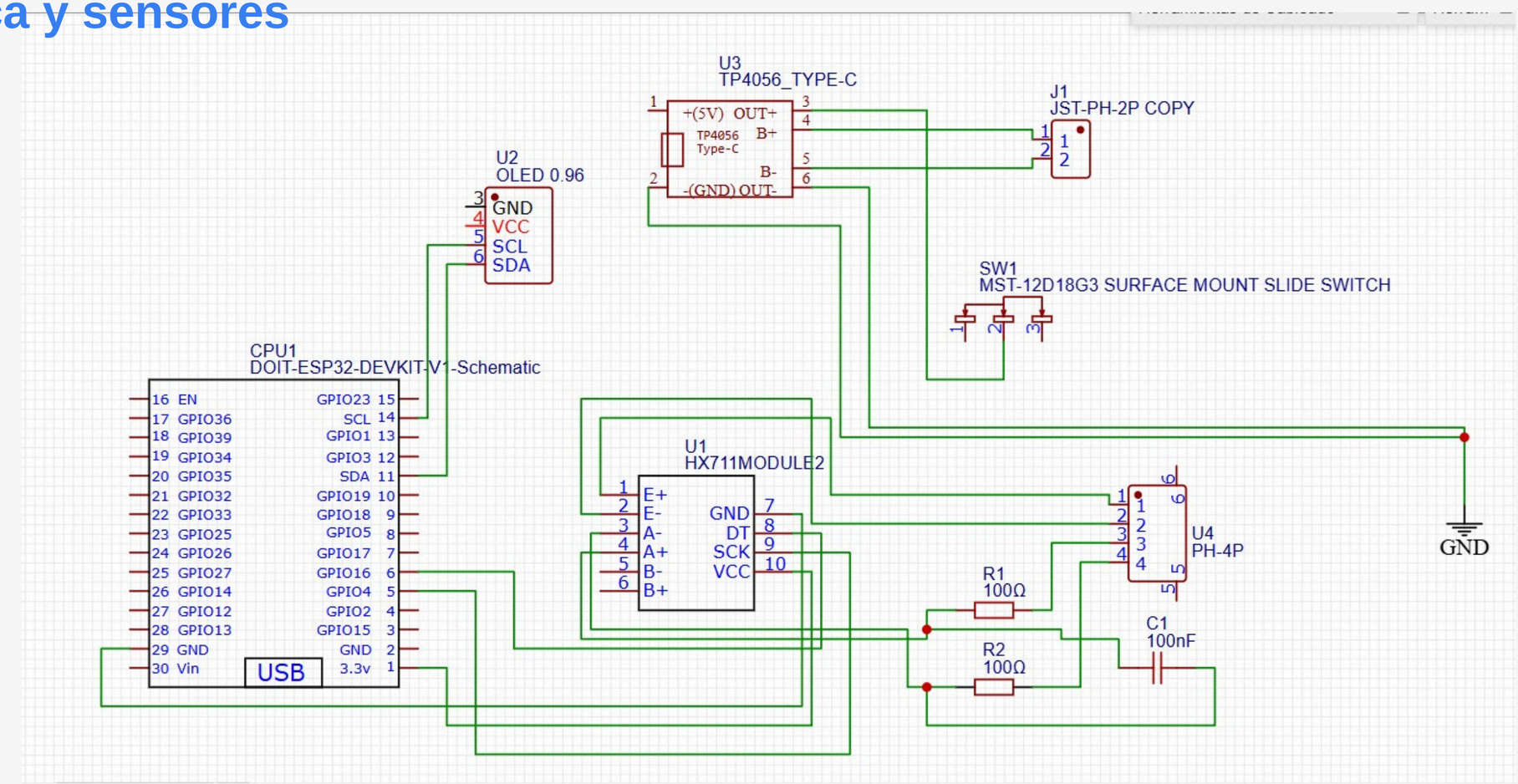
Conector JST-PH de 2 pines



Interruptor deslizante (SW1)

Implementación preliminar de los subsistemas

Electronica y sensores



Implementación preliminar de los subsistemas

Programación y control

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "HX711.h"
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>

// ========================================
// CONFIGURACIÓN DE PANTALLA OLED
// ========================================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ========================================
// PINES Y HARDWARE (ESP32)
// ========================================
const int HX711_DOUT_PIN = 4;  // Pin GPIO 4 del ESP32
const int HX711_SCK_PIN  = 5;  // Pin GPIO 5 del ESP32

HX711 scale;

// Factor de calibración (ajustar según tu celda tipo botón)
float calibration_factor = 2280.0;

// Geometría de la Valva Sternal en pulgadas cuadradas (sq in)
const float AREA_VALVA_SQ_IN = 3.875;

// ========================================
// CONFIGURACIÓN BLE (ESP32)
// ========================================
#define SERVICE_UUID        "19B10000-E8F2-537E-4F6C-D104768A1214"
#define CHARACTERISTIC_UUID "19B10001-E8F2-537E-4F6C-D104768A1214"
```



Implementación preliminar de los subsistemas

Programación y control

```
BLECharacteristic *pCharacteristic;
bool deviceConnected = false;

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    }
    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};

unsigned long lastSendTime = 0;
const long interval = 500; // Actualizar cada 500 ms

void setup() {
  Serial.begin(115200);

  // Inicializar Pantalla OLED (Dirección I2C 0x3C)
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("Fallo al iniciar la pantalla OLED"));
    while(1);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,0);
  display.print("Iniciando Pectus...");
  display.display();
  delay(1000);

  // Inicializar Celda de Carga
  scale.begin(HX711_DOUT_PIN, HX711_SCK_PIN);
  scale.set_scale(calibration_factor);
  scale.tare(); // Poner a cero
```

```
  // Inicializar BLE en ESP32
  BLEDevice::init("PectusBrace_ESP32");
  BLEServer *pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService = pServer->createService(SERVICE_UUID);
  pCharacteristic = pService->createCharacteristic(
                      CHARACTERISTIC_UUID,
                      BLECharacteristic::PROPERTY_READ   |
                      BLECharacteristic::NOTIFY
                    );
  pService->start();

  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pServer->startAdvertising();

  Serial.println("ESP32 BLE Servidor listo, esperando conexion...");
}

void loop() {
  unsigned long currentTime = millis();

  if (currentTime - lastSendTime >= interval) {
    lastSendTime = currentTime;

    // 1. Leer fuerza en Libras (lbs)
    float fuerza_lbs = scale.get_units(5);
    if (fuerza_lbs < 0) fuerza_lbs = 0.0;

    // 2. Calcular Presión en PSI
    float presion_psi = fuerza_lbs / AREA_VALVA_SQ_IN;
```

```
    // 3. Mostrar datos en la Pantalla OLED
    display.clearDisplay();
    display.setCursor(0, 0);
    display.setTextSize(1);
    display.print("Corse Pectus Care");
    display.setCursor(0, 16);
    display.print("Fuerza: ");
    display.print(fuerza_lbs, 1);
    display.print(" lbs");

    display.setCursor(0, 32);
    display.setTextSize(2);
    display.print(presion_psi, 1);
    display.print(" PSI");

    display.setCursor(0, 52);
    display.setTextSize(1);
    if (deviceConnected) {
      display.print("Estado: Conectado");
    } else {
      display.print("Estado: Esperando...");
    }
    display.display();

    // 4. Enviar datos por Bluetooth si hay conexión
    if (deviceConnected) {
      char txString[8];
      dtostrf(presion_psi, 1, 2, txString);
      pCharacteristic->setValue(txString);
      pCharacteristic->notify();
    }
```

```
    // Monitor Serie
    Serial.print("Fuerza: "); Serial.print(fuerza_lbs); Serial.print(" lbs | ");
    Serial.print("Presion: "); Serial.print(presion_psi); Serial.println(" PSI");
  }
}
```

Gracias!

